

# Deep Learning

## CSE641/ECE555

Lecture 11 | Take your own notes during lectures

Vinayak Abrol <abrol@iiitd.ac.in>



INDRAPRASTHA INSTITUTE of  
INFORMATION TECHNOLOGY DELHI

# Generative Adversarial Networks (GANs)

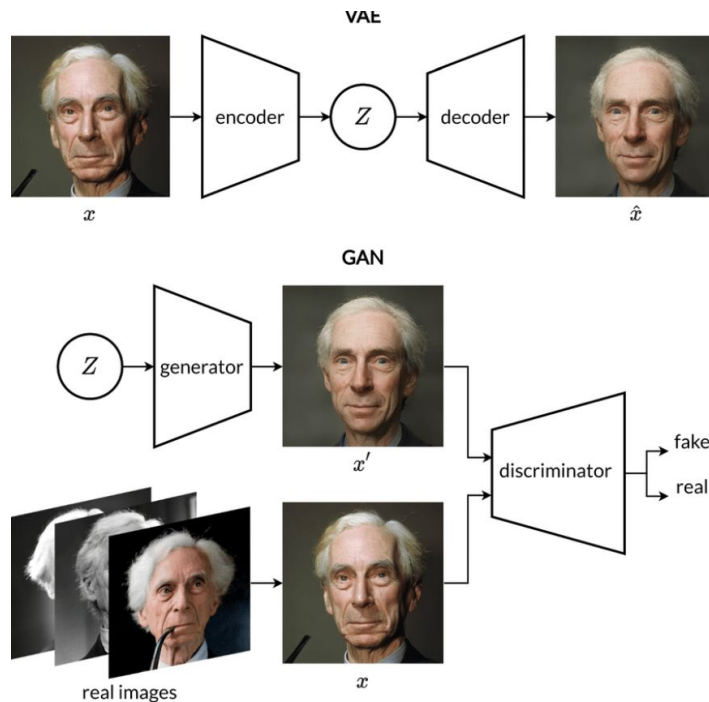
GANs extend the idea of VAEs to model/learn densities in high-dimensions.

Like VAE we assume a prior distribution from which we generate examples directly via Generator

There is no encoder.

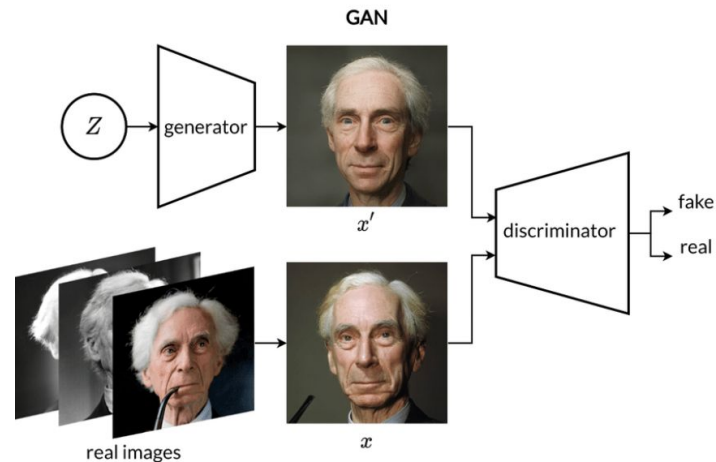
Instead we have a Discriminator that measures the quality of generator.

The approach is adversarial since the two networks have antagonistic objectives.



# Generative Adversarial Networks (GANs)

- The role of the **Discriminator** is to detect if a sample is **Real** or **Fake**.
- The role of the **Generator** is to 'fool the **Discriminator**'; produce realistic samples: given some random noise following a **fixed and simple distribution**.
- Discriminator is optimized to minimize a standard classification loss.
- Generator is optimized to maximize that loss.



## Important:

Generator maximize the loss **through** the Discriminator. Hence the backward pass will propagate the gradient of the loss through the Discriminator to the Generator.

This ensures the Generator will be **constantly updated** during training to remove any statistical structures learned **by the discriminator from synthetic samples**.

# GANs: Mathematical Model

Consider the binary classification problem Real/Fake

$$\text{Data} = [(x_1, 1) \dots (x_N, 1); (G(Z_1), 0) \dots (G(Z_N), 0)]$$

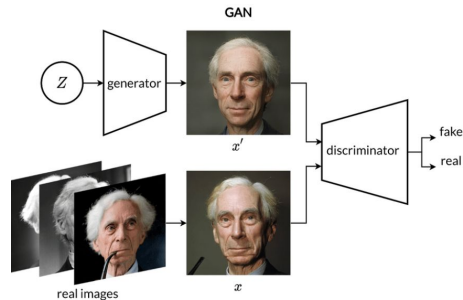
Real Samples

Fake Samples

$$z_n \sim \mathcal{N}(0, \mathbf{I})$$

Discriminator Loss:

Generator Loss:



# GANs: Mathematical Model

Consider the binary classification problem Real/Fake

$$\text{Data} = [(x_1, 1) \dots (x_N, 1); (G(Z_1), 0) \dots (G(Z_N), 0)]$$

Real Samples

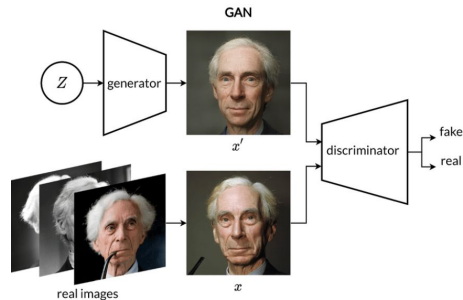
Fake Samples

$$z_n \sim \mathcal{N}(0, \mathbf{I})$$

$$\mathcal{L} = -(y \log \hat{y} + (1 - y) \log(1 - \hat{y}))$$

Discriminator Loss:

Generator Loss:



# GANs: Mathematical Model

Consider the binary classification problem Real/Fake

$$\text{Data} = [(x_1, 1) \dots (x_N, 1); (G(Z_1), 0) \dots (G(Z_N), 0)]$$

Real Samples

Fake Samples

$$z_n \sim \mathcal{N}(0, \mathbf{I})$$

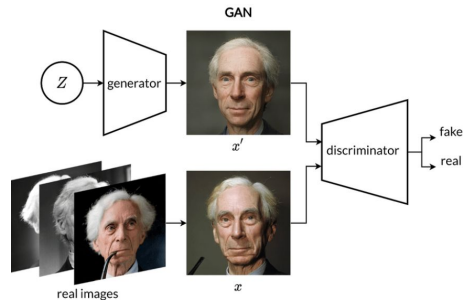
$$\mathcal{L} = -(y \log \hat{y} + (1 - y) \log(1 - \hat{y}))$$

Discriminator Loss:

$$\mathcal{L}(D) = -(1 \log(\mathbf{D}(x)) + (1 - 1) \log(1 - \mathbf{D}(x))) = -\log(\mathbf{D}(x))$$

$$\mathcal{L}(D) = -(0 \log(\mathbf{D}(x)) + (1 - 0) \log(1 - \mathbf{D}(x))) = -\log(1 - \mathbf{D}(x))$$

Generator Loss:



# GANs: Mathematical Model

Consider the binary classification problem Real/Fake

$$\text{Data} = [(x_1, 1) \dots (x_N, 1); (G(Z_1), 0) \dots (G(Z_N), 0)]$$

Real Samples

Fake Samples

$$z_n \sim \mathcal{N}(0, \mathbf{I})$$

$$\mathcal{L} = -(y \log \hat{y} + (1 - y) \log(1 - \hat{y}))$$

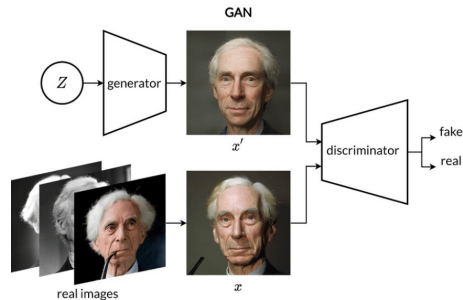
Discriminator Loss:

$$\mathcal{L}(D) = -(1 \log(\mathbf{D}(x)) + (1 - 1) \log(1 - \mathbf{D}(x))) = -\log(\mathbf{D}(x))$$

$$\mathcal{L}(D) = -(0 \log(\mathbf{D}(\mathbf{G}(z))) + (1 - 0) \log(1 - \mathbf{D}(\mathbf{G}(z)))) = -\log(1 - \mathbf{D}(\mathbf{G}(z)))$$

$$D = \min [ -(\log(\mathbf{D}(x)) + \log(1 - \mathbf{D}(x))) ] = \max [ \log(\mathbf{D}(x)) + \log(1 - \mathbf{D}(x)) ]$$

Generator Loss:



# GANs: Mathematical Model

Consider the binary classification problem Real/Fake

$$\text{Data} = [(x_1, 1) \dots (x_N, 1); (G(Z_1), 0) \dots (G(Z_N), 0)]$$

Real Samples

Fake Samples

$$z_n \sim \mathcal{N}(0, \mathbf{I})$$

$$\mathcal{L} = -(y \log \hat{y} + (1 - y) \log(1 - \hat{y}))$$

Discriminator Loss:

$$\mathcal{L}(D) = -(1 \log(\mathbf{D}(x)) + (1 - 1) \log(1 - \mathbf{D}(x))) = -\log(\mathbf{D}(x))$$

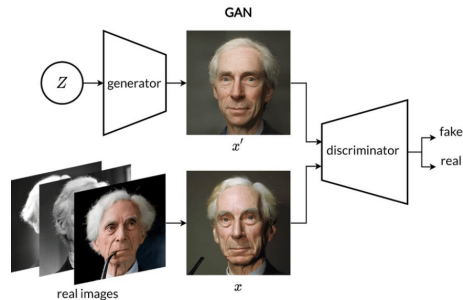
$$\mathcal{L}(D) = -(0 \log(\mathbf{D}(\mathbf{G}(z))) + (1 - 0) \log(1 - \mathbf{D}(\mathbf{G}(z)))) = -\log(1 - \mathbf{D}(\mathbf{G}(z)))$$

$$D = \min [-(\log(\mathbf{D}(x)) + \log(1 - \mathbf{D}(x)))] = \max [\log(\mathbf{D}(x)) + \log(1 - \mathbf{D}(x))]$$

Generator Loss:

$$G = \min [\log(1 - \mathbf{D}(G(z)))]$$

Discriminator via generator only gets fake data





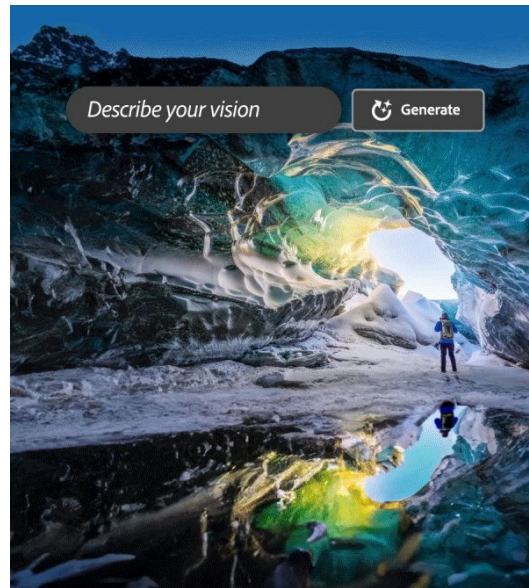
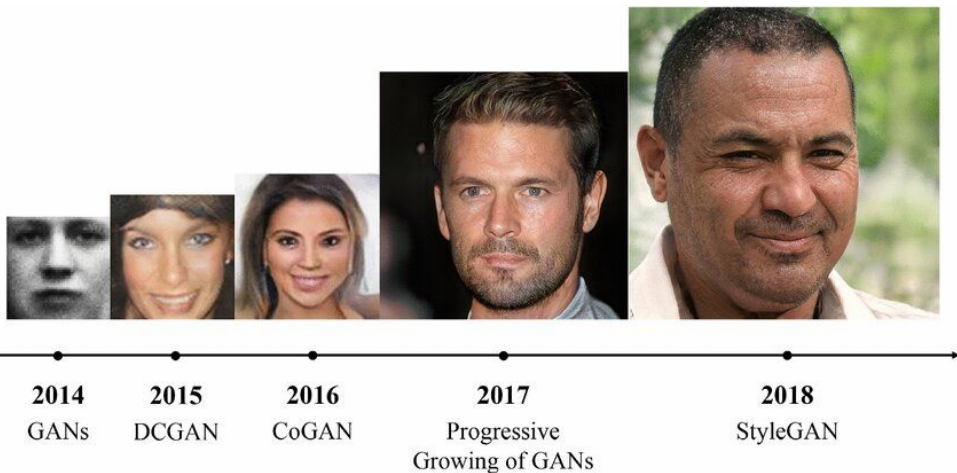
# GANs: Mathematical Model

## GAN Loss (Min-Max Game)

Find a G whose loss against its best adversary D is low

$$GAN : \min_G \max_D [\log(\mathbf{D}(x)) + \log(1 - \mathbf{D}(x))]$$

$$\min_G \max_D \mathbb{E}_{x \sim \mathcal{D}} [\log(\mathbf{D}(x))] + \mathbb{E}_{x \sim \mathcal{G}} [\log(1 - \mathbf{D}(x))]$$



# GANs: Mathematical Model

---

GAN Loss (How to Optimize?)

Find a G whose loss against its best adversary D is low

$$\min_G \max_D \mathcal{L}_{GAN}$$

$$\mathbb{E}_{x \sim \mathcal{D}} [\log(\mathbf{D}(x))] + \mathbb{E}_{x \sim \mathcal{G}} [\log(1 - \mathbf{D}(x))]$$

Fix G & maximize

# GANs: Mathematical Model

---

GAN Loss (How to Optimize?)

Find a G whose loss against its best adversary D is low

$$\min_G \max_D \mathcal{L}_{GAN}$$

$$\mathbb{E}_{x \sim \mathcal{D}} [\log(\mathbf{D}(x))] + \mathbb{E}_{x \sim \mathcal{G}} [\log(1 - \mathbf{D}(x))]$$

Fix G & maximize

$$\int_x p(x) \log(\mathbf{D}(x)) + q(x) \log(1 - \mathbf{D}(x)) \, dx$$

# GANs: Mathematical Model

GAN Loss (How to Optimize?)

Find a G whose loss against its best adversary D is low

$$\min_G \max_D \mathcal{L}_{GAN}$$

$$\mathbb{E}_{x \sim \mathcal{D}} [\log(\mathbf{D}(x))] + \mathbb{E}_{x \sim \mathcal{G}} [\log(1 - \mathbf{D}(x))]$$

Fix G & maximize  $\int_x p(x) \log(\mathbf{D}(x)) + q(x) \log(1 - \mathbf{D}(x)) \, dx$

if  $y = a \log(y) + b \log(1 - y)$ , the optimal y is

$$\Rightarrow y^* = \frac{a}{a + b}$$

$$y = a \log(y) + b \log(1 - y)$$

$$y' = \frac{a}{y} - \frac{b}{1 - y}$$

$$\frac{a}{y^*} = \frac{b}{1 - y^*}$$

$$\frac{1 - y^*}{y^*} = \frac{b}{a}$$

$$\frac{1}{y^*} = \frac{a + b}{a}$$

$$y^* = \frac{a}{a + b}$$

Find optimal  $y^*$  by setting  $y' = 0$ .

# GANs: Mathematical Model

GAN Loss (How to Optimize?)

Find a G whose loss against its best adversary D is low

$$\min_G \max_D \mathcal{L}_{GAN}$$

$$\mathbb{E}_{x \sim \mathcal{D}} [\log(\mathbf{D}(x))] + \mathbb{E}_{x \sim \mathcal{G}} [\log(1 - \mathbf{D}(x))]$$

Fix G & maximize  $\int_x p(x) \log(\mathbf{D}(x)) + q(x) \log(1 - \mathbf{D}(x)) \, dx$

$$\mathbb{E}_{x \sim \mathcal{D}} \left[ \log \left( \frac{p(x)}{p(x) + q(x)} \right) \right] + \mathbb{E}_{x \sim \mathcal{G}} \left[ \log \left( \frac{q(x)}{p(x) + q(x)} \right) \right]$$

if  $y = a \log(y) + b \log(1 - y)$ , the optimal y is

$$\Rightarrow y^* = \frac{a}{a + b}$$

# GANs: Mathematical Model

GAN Loss (How to Optimize?)

Find a G whose loss against its best adversary D is low

$$\min_G \max_D \mathcal{L}_{GAN}$$

Fix G & maximize 
$$\int_x p(x) \log(\mathbf{D}(x)) + q(x) \log(1 - \mathbf{D}(x)) \, dx$$

if  $y = a \log(y) + b \log(1 - y)$ , the optimal y is  
 $\Rightarrow y^* = \frac{a}{a + b}$

$$\mathbb{E}_{x \sim \mathcal{D}} \left[ \log \left( \frac{p(x)}{p(x) + q(x)} \right) \right] + \mathbb{E}_{x \sim \mathcal{G}} \left[ \log \left( \frac{q(x)}{p(x) + q(x)} \right) \right]$$
$$\text{KL} \left[ p(x) \parallel \frac{p(x) + q(x)}{2} \right] + \text{KL} \left[ q(x) \parallel \frac{p(x) + q(x)}{2} \right] - 2 \log 2$$

Now minimise for G

Using the definition of Jensen-Shannon Divergence

$$D_{JS}(p \parallel q) = \frac{1}{2} D_{KL}(p \parallel \frac{p+q}{2}) + \frac{1}{2} D_{KL}(q \parallel \frac{p+q}{2})$$

# GANs: Mathematical Model

GAN Loss (How to Optimize?)

Find a G whose loss against its best adversary D is low

$$\min_G \max_D \mathcal{L}_{GAN}$$

Fix G & maximize 
$$\mathbb{E}_{x \sim \mathcal{D}} [\log(\mathbf{D}(x))] + \mathbb{E}_{x \sim \mathcal{G}} [\log(1 - \mathbf{D}(x))] \\ \int_x p(x) \log(\mathbf{D}(x)) + q(x) \log(1 - \mathbf{D}(x)) \, dx$$

if  $y = a \log(y) + b \log(1 - y)$ , the optimal y is  
 $\Rightarrow y^* = \frac{a}{a + b}$

$$\mathbb{E}_{x \sim \mathcal{D}} \left[ \log \left( \frac{p(x)}{p(x) + q(x)} \right) \right] + \mathbb{E}_{x \sim \mathcal{G}} \left[ \log \left( \frac{q(x)}{p(x) + q(x)} \right) \right] \\ \text{KL} \left[ p(x) \parallel \frac{p(x) + q(x)}{2} \right] + \text{KL} \left[ q(x) \parallel \frac{p(x) + q(x)}{2} \right] - 2 \log 2$$

Now minimise for G 
$$2 D_{JS}(p, q) - 2 \log 2$$

Using the definition of Jensen-Shannon Divergence

# GANs: Mathematical Model

GAN Loss (How to Optimize?)

Find a G whose loss against its best adversary D is low

$$\min_G \max_D \mathcal{L}_{GAN}$$

Optimality will occur when both p & q will match

$$p = q; \mathbf{D}^* = \frac{p}{p + q} = .5$$

$$\min \max \mathcal{L} = -2 \log 2$$

– y), the optimal y is

$$\text{KL}[p(x) || \frac{p(x) + q(x)}{2}] + \text{KL}[q(x) || \frac{p(x) + q(x)}{2}] - 2 \log 2$$

Now minimise for G

$$2 D_{JS}(p, q) - 2 \log 2$$

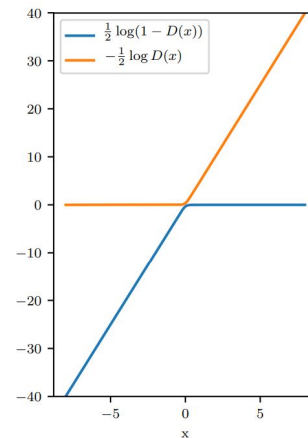
Using the definition of Jensen-Shannon Divergence



# GANs: Mathematical Model

## Enhancing Generator

- Early in learning, when  $G$  is poor,  $D$  can reject samples with high confidence because they are clearly different from the training data.
- $\log(1 - D(G(z)))$  saturates, since  $D(G(z)) \sim 0$
- Rather than training  $G$  to minimize  $\log(1 - D(G(z)))$ 
  - we can train  $G$  to maximize  $\log D(G(z))$
- This is the same as saying that instead of training the Generator to minimize the probability of the image being fake, it is going to maximize the probability of the image being real.



## Total Loss Function

$$\begin{aligned} T_{loss} &= -\frac{1}{n} \cdot \sum_i^n \log(D(x_i)) + \log(1 - D(G(z))) - \frac{1}{n} \cdot \sum_i^n \log(D(G(z))) = \\ &= -\frac{1}{n} \cdot \sum_i^n \log(D(x_i)) + \log(1 - D(G(z))) + \log(D(G(z))) \end{aligned}$$

# Convolutional GANs

GANs were initially difficult to optimize

Radford et al. converged to the following rules:

- Replace pooling layers with strided convolutions in D and strided transposed convolutions in G
- use batchnorm in both D and G
- remove fully connected hidden layers
- use ReLU in G except for the output, which uses Tanh
- use LeakyReLU activation in D for all layers.

Training a standard GAN still can show:

- oscillations without convergence.
- “mode collapse”, when G models very well a small sub-population e.g., only cat class

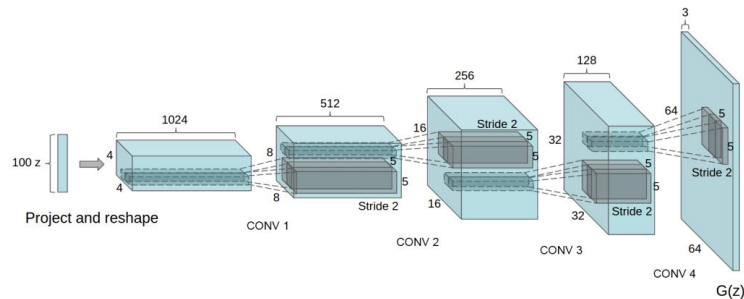


Figure 1: DCGAN generator used for LSUN scene modeling. A 100 dimensional uniform distribution  $Z$  is projected to a small spatial extent convolutional representation with many feature maps. A series of four fractionally-strided convolutions (in some recent papers, these are wrongly called deconvolutions) then convert this high level representation into a  $64 \times 64$  pixel image. Notably, no fully connected or pooling layers are used.

Reference implementation:

[https://pytorch.org/tutorials/beginner/dcgan\\_faces\\_tutorial.html](https://pytorch.org/tutorials/beginner/dcgan_faces_tutorial.html)

<https://github.com/pytorch/examples.git>

# Reading

---

1. A. Brock, J. Donahue, and K. Simonyan. Large scale GAN training for high fidelity natural image synthesis. CoRR, abs/1809.11096, 2018.
2. I. J. Goodfellow, J. Pouget-Abadie, M. Mirza, B. Xu, D. Warde-Farley, S. Ozair, A. Courville, and Y. Bengio. Generative adversarial networks. CoRR, abs/1406.2661, 2014.
3. M. Heusel, H. Ramsauer, T. Unterthiner, B. Nessler, and S. Hochreiter. GANs trained by a two time-scale update rule converge to a local nash equilibrium. CoRR, abs/1706.08500, 2017.
4. T. Karras, S. Laine, and T. Aila. A style-based generator architecture for generative adversarial networks. CoRR, abs/1812.04948, 2018.
5. A. Radford, L. Metz, and S. Chintala. Unsupervised representation learning with deep convolutional generative adversarial networks. CoRR, abs/1511.06434, 2015.
6. T. Salimans, I. Goodfellow, W. Zaremba, V. Cheung, A. Radford, X. Chen, and X. Chen. Improved techniques for training GANs. In Neural Information Processing Systems (NIPS), pages 2234-2242, 2016.
7. C. Szegedy, V. Vanhoucke, S. Ioffe, J. Shlens, and Z. Wojna. Rethinking the inception architecture for computer vision. CoRR, abs/1512.00567, 2015.

# Matching Distributions

---

Arjovsky et al. showed the pitfalls of KL and JS divergence with a simple example.

Generalized idea: JS does not account much for the metric structure of the space. [More in Learning Theory course]

Example:

Consider two parallel lines at a distance of one ( $\theta=1$ ) where one line is at  $x=0$  [P] and another at  $x=1$  [Q].

Aim is to match Q with reference distribution P.

Ideally as  $\theta$  decreases Q and P should match.

Let's see if this happens?

$$D_{KL}(p||q) = \int p(x) \log \frac{p(x)}{q(x)}$$

$$D_{KL}(p||q) = \sum p(x) \log \frac{p(x)}{q(x)}$$

$$D_{JS}(p||q) = \frac{1}{2} D_{KL}(p||\frac{p+q}{2}) + \frac{1}{2} D_{KL}(q||\frac{p+q}{2})$$

# Matching Distributions

Arjovsky et al. showed the pitfalls of KL and JS divergence with a simple example.

Generalized idea: JS does not account much for the metric structure of the space. [More in Learning Theory course]

Example:

Consider two parallel lines at a distance of one ( $\theta=1$ ) where one line is at  $x=0$  [P] and another at  $x=1$  [Q].

Aim is to match Q with reference distribution P.

Ideally as  $\theta$  decreases Q and P should match.  
Let's see if this happens?

$$D_{KL}(p||q) = \int p(x) \log \frac{p(x)}{q(x)}$$

$$D_{KL}(p||q) = \sum p(x) \log \frac{p(x)}{q(x)}$$

$$D_{JS}(p||q) = \frac{1}{2} D_{KL}(p||\frac{p+q}{2}) + \frac{1}{2} D_{KL}(q||\frac{p+q}{2})$$

$$KL(P||Q) = P(x=0) \log \frac{P(x=0)}{Q(x=0)} + P(x=1) \log \frac{P(x=1)}{Q(x=1)}$$

$$KL(P||Q) = 1 \log \frac{1}{0} + 0 \log \frac{1}{1} = \infty$$

$$KL(Q||P) = Q(x=0) \log \frac{Q(x=0)}{P(x=0)} + Q(x=1) \log \frac{Q(x=1)}{P(x=1)}$$

$$KL(Q||P) = 0 \log \frac{0}{1} + 1 \log \frac{1}{0} = \infty$$

# Matching Distributions

Arjovsky et al. showed the pitfalls of KL and JS divergence with a simple example.

Generalized idea: JS does not account much for the metric structure of the space. [More in Learning Theory course]

Example:

Consider two parallel lines at a distance of one ( $\theta=1$ ) where one line is at  $x=0$  [P] and another at  $x=1$  [Q].

Aim is to match Q with reference distribution P.

Ideally as  $\theta$  decreases Q and P should match.

Let's see if this happens?

$$D_{KL}(p||q) = \int p(x) \log \frac{p(x)}{q(x)}$$

$$D_{KL}(p||q) = \sum p(x) \log \frac{p(x)}{q(x)}$$

$$D_{JS}(p||q) = \frac{1}{2} D_{KL}(p||\frac{p+q}{2}) + \frac{1}{2} D_{KL}(q||\frac{p+q}{2})$$

$$JS(P||Q) = \frac{1}{2} [P(x=0) \log \frac{P(x=0)}{\frac{P(x=0)+Q(x=0)}{2}} + P(x=1) \log \frac{P(x=1)}{\frac{P(x=1)+Q(x=1)}{2}} + Q(x=0) \log \frac{Q(x=0)}{\frac{P(x=0)+Q(x=0)}{2}} + Q(x=1) \log \frac{Q(x=1)}{\frac{P(x=1)+Q(x=1)}{2}}]$$

$$JS(P||Q) = \frac{1}{2} [1 \log(\frac{1}{1+0}) + 0 + 0 + 1 \log \frac{1}{1+0}]$$

$$JS(P||Q) = \frac{1}{2} [2 \log 2]$$

$$JS(P||Q) = \log 2$$

# Matching Distributions

Arjovsky et al. showed the pitfalls of KL and JS divergence with a simple example.

Generalized idea: JS does not account much for the metric structure of the space. [More in Learning Theory course]

Example:

Consider two parallel lines at a distance of one ( $\theta=1$ ) where one line is at  $x=0$  [P] and another at  $x=1$  [Q].

Aim is to match Q with reference distribution P.

Ideally as  $\theta$  decreases Q and P should match.

Let's see if this happens?

<https://lilianweng.github.io/posts/2017-08-20-gan/>

$$D_{KL}(p||q) = \int p(x) \log \frac{p(x)}{q(x)}$$

$$D_{KL}(p||q) = \sum p(x) \log \frac{p(x)}{q(x)}$$

$$D_{JS}(p||q) = \frac{1}{2} D_{KL}(p||\frac{p+q}{2}) + \frac{1}{2} D_{KL}(q||\frac{p+q}{2})$$

$$KL(P||Q) = KL(Q||P) = \begin{cases} \infty & \text{if } \theta \neq 0 \\ 0 & \text{if } \theta = 0 \end{cases}$$

$$JS(P||Q) = \begin{cases} \log 2 & \text{if } \theta \neq 0 \\ 0 & \text{if } \theta = 0 \end{cases}$$

The difference between distributions is same irrespective of  $\theta$  (except at  $\theta=0$ )

If GAN is parameterized by  $\Phi$  then  $\theta = f(\Phi)$  i.e., as parameters  $\Phi$  changes,  $\theta$  should change?

But gradient will be zero

$$\theta = f(\phi) \\ \frac{dL}{d\phi} = \frac{dL}{d\theta} \frac{d\theta}{d\phi}$$

# Wasserstein GAN (WGAN)

## Wasserstein Distance

- Earth Moving Distance
- minimum mass displacement to transform one distribution into the other

Let's take 4 parallel lines

Strategy 1:

Shift 0.2 from  $x=2$  to  $x=0$

Shift 0.1 from  $x=3$  to  $x=0$

Shift 0.7 from  $x=3$  to  $x=1$

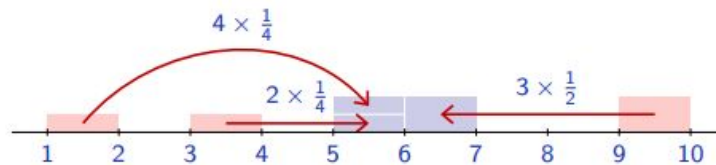
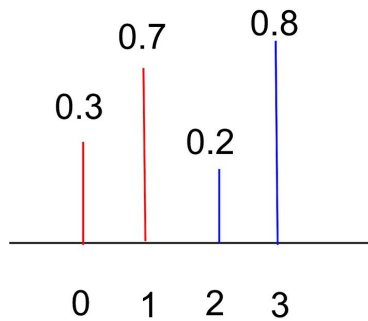
Strategy 2:

Shift 0.1 from  $x=2$  to  $x=0$

Shift 0.1 from  $x=2$  to  $x=1$

Shift 0.2 from  $x=3$  to  $x=0$

Shift 0.6 from  $x=3$  to  $x=0$



$$\mu = \frac{1}{4}\mathbf{1}_{[1,2]} + \frac{1}{4}\mathbf{1}_{[3,4]} + \frac{1}{2}\mathbf{1}_{[9,10]} \quad \mu' = \frac{1}{2}\mathbf{1}_{[5,7]}$$

$$W(\mu, \mu') = 4 \times \frac{1}{4} + 2 \times \frac{1}{4} + 3 \times \frac{1}{2} = 3$$

## Wasserstein Distance

(mass multiplied by shift distance: Transport Plan)

$$S1: .2 \times 2 + .1 \times 3 + .7 \times 2 = 2.1$$

$$S2: .1 \times 2 + .2 \times 1 + .2 \times 3 + .6 \times 3 = 2.8$$

WD or EMD is proportional to  $\theta$



# Wasserstein GAN (WGAN)

---

$$W(p_r, p_g) = \inf_{\gamma \sim \Pi(p_r, p_g)} \mathbb{E}_{(x, y) \sim \gamma} [\|x - y\|]$$

Here,  $\Pi()$  is the set of all possible joint probability distributions (corresponding to possible Transport Plans)

Infimum: (also known as greatest lower bound) indicates that we are only interested in the smallest cost.

$$W(p_r, p_g) = \frac{1}{K} \sup_{\|f\|_L \leq K} \mathbb{E}_{x \sim p_r} [f(x)] - \mathbb{E}_{x \sim p_g} [f(x)]$$

Using Kantorovich-Rubinstein duality

Important:  $f$  is a  $K$ -Lipschitz function following the constraint:  $|f(x_1) - f(x_2)| \leq K|x_1 - x_2|$

Like other deep learning problems, we can build a deep network to learn it

## WGAN

Discriminator  $D()$  now called the critic network. It outputs a scalar score rather than a probability (no sigmoid)

# Wasserstein GAN (WGAN)

$$\min_G \max_D \mathbb{E}_{x \sim \mathcal{D}} [\log(D(x))] + \mathbb{E}_{x \sim \mathcal{G}} [\log(1 - D(G(z)))]$$

$$\min_G \max_{|D| \leq 1} \mathbb{E}_{x \sim \mathcal{D}} [D(x)] - \mathbb{E}_{x \sim \mathcal{G}} [D(G(z))]$$

Arjovsky et al. achieved ID<1 this by clipping D's weights

A greater learning stability addresses “mode collapse” problem.

A greater interpretability of the loss, helps as a better indicator of the quality of generation.

---

**Algorithm 1** WGAN, our proposed algorithm. All experiments in the paper used the default values  $\alpha = 0.00005$ ,  $c = 0.01$ ,  $m = 64$ ,  $n_{\text{critic}} = 5$ .

---

**Require:** :  $\alpha$ , the learning rate.  $c$ , the clipping parameter.  $m$ , the batch size.  $n_{\text{critic}}$ , the number of iterations of the critic per generator iteration.

**Require:** :  $w_0$ , initial critic parameters.  $\theta_0$ , initial generator's parameters.

```
1: while  $\theta$  has not converged do
2:   for  $t = 0, \dots, n_{\text{critic}}$  do
3:     Sample  $\{x^{(i)}\}_{i=1}^m \sim \mathbb{P}_r$  a batch from the real data.
4:     Sample  $\{z^{(i)}\}_{i=1}^m \sim p(z)$  a batch of prior samples.
5:      $g_w \leftarrow \nabla_w [\frac{1}{m} \sum_{i=1}^m f_w(x^{(i)}) - \frac{1}{m} \sum_{i=1}^m f_w(g_\theta(z^{(i)}))]$ 
6:      $w \leftarrow w + \alpha \cdot \text{RMSPProp}(w, g_w)$ 
7:      $w \leftarrow \text{clip}(w, -c, c)$ 
8:   end for
9:   Sample  $\{z^{(i)}\}_{i=1}^m \sim p(z)$  a batch of prior samples.
10:   $g_\theta \leftarrow -\nabla_\theta \frac{1}{m} \sum_{i=1}^m f_w(g_\theta(z^{(i)}))$ 
11:   $\theta \leftarrow \theta - \alpha \cdot \text{RMSPProp}(\theta, g_\theta)$ 
12: end while
```

---

# Wasserstein GAN (WGAN)

$$\min_G \max_D \mathbb{E}_{x \sim \mathcal{D}} [\log(D(x))] + \mathbb{E}_{x \sim \mathcal{G}} [\log(1 - D(G(z)))]$$

$$\min_G \max_{|D| \leq 1} \mathbb{E}_{x \sim \mathcal{D}} [D(x)] - \mathbb{E}_{x \sim \mathcal{G}} [D(G(z))]$$

Arjovsky et al. achieved  $ID < 1$  this by clipping  $D$ 's weights

A greater learning stability addresses “mode collapse” problem.

A greater interpretability of the loss, helps as a better indicator of the quality of generation.

Nothing is perfect: Arjovsky et al. mentioned Weight clipping is a clearly terrible way to enforce a Lipschitz constraint” (Oops!).

– WGAN



– Standard GAN



Without BN

# Wasserstein GAN (WGAN)

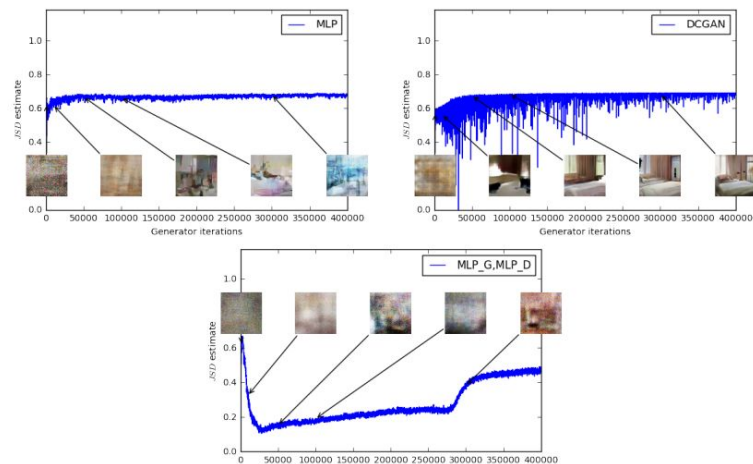


Figure 4: JS estimates for an MLP generator (upper left) and a DCGAN generator (upper right) trained with the standard GAN procedure. Both had a DCGAN discriminator. Both curves have increasing error. Samples get better for the DCGAN but the JS estimate increases or stays constant, pointing towards no significant correlation between sample quality and loss. Bottom: MLP with both generator and discriminator. The curve goes up and down regardless of sample quality. All training curves were passed through the same median filter as in Figure 3.

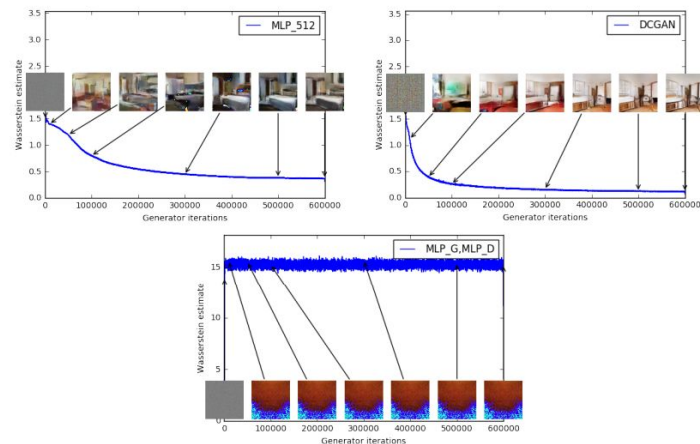


Figure 3: Training curves and samples at different stages of training. We can see a clear correlation between lower error and better sample quality. Upper left: the generator is an MLP with 4 hidden layers and 512 units at each layer. The loss decreases consistently as training progresses and sample quality increases. Upper right: the generator is a standard DCGAN. The loss decreases quickly and sample quality increases as well. In both upper plots the critic is a DCGAN without the sigmoid so losses can be subjected to comparison. Lower half: both the generator and the discriminator are MLPs with substantially high learning rates (so training failed). Loss is constant and samples are constant as well. The training curves were passed through a median filter for visualization purposes.

M. Arjovsky, S. Chintala, and L. Bottou. Wasserstein GAN. CoRR, abs/1701.07875, 2017.

T. Miyato, T. Kataoka, M. Koyama, and Y. Yoshida. Spectral normalization for generative adversarial networks. In International Conference on Learning Representations (ICLR), 2018.

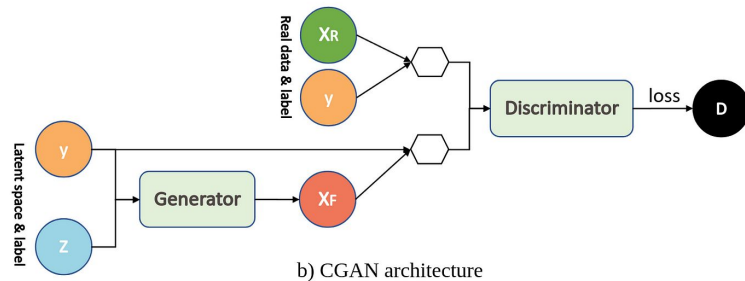
Gulrajani, Ishaan, Faruk Ahmed, Martin Arjovsky, Vincent Dumoulin, and Aaron C. Courville. "Improved training of wasserstein gans." Advances in neural information processing systems 30 (2017).

# Conditional GAN

$$\min_G \max_D \mathbb{E}_{(x,y) \sim \mathcal{D}} [\log(D(x, y))] + \mathbb{E}_{(x,y) \sim \mathcal{G}} [\log(1 - D(G(z), y))]$$

The task here goes beyond sampling according to a certain distribution! One must have a way to condition the distribution according to an input signal and type of task e.g., next frame prediction, in-painting, segmentation, style transfer.

1. Condition over class labels (Mirza and Osindero, 2014)
2. Condition BN param over class labels (Brock et al., 2018)
3. Pixel-to-pixel correspondence (Isola et al. 2016)
4. Image-to-image correspondence (Zhu et al. 2017)





# Conditional GAN

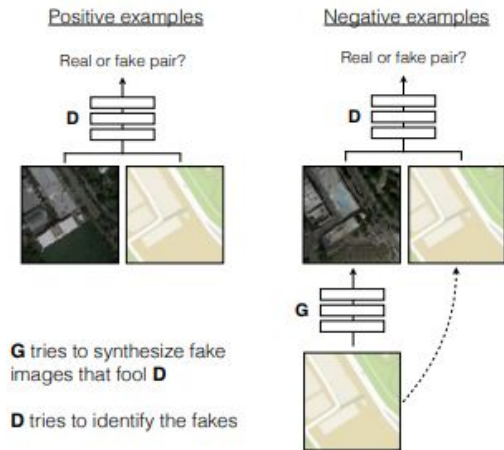


Figure 2: Training a conditional GAN to predict aerial photos from maps. The discriminator,  $D$ , learns to classify between real and synthesized pairs. The generator learns to fool the discriminator. Unlike an unconditional GAN, both the generator and discriminator observe an input image.

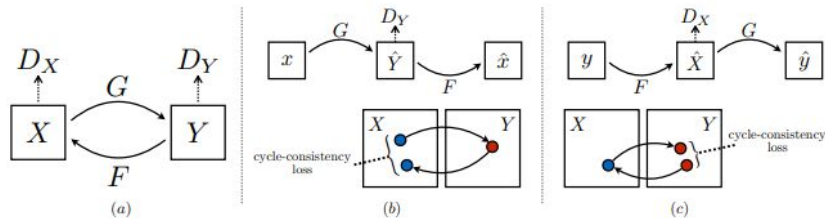
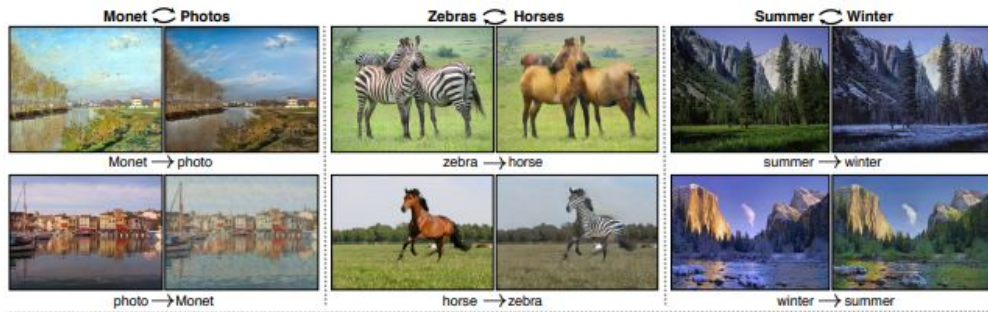
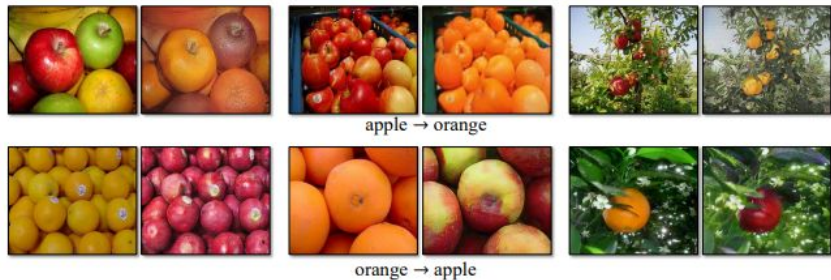


Figure 3: (a) Our model contains two mapping functions  $G : X \rightarrow Y$  and  $F : Y \rightarrow X$ , and associated adversarial discriminators  $D_Y$  and  $D_X$ .  $D_Y$  encourages  $G$  to translate  $X$  into outputs indistinguishable from domain  $Y$ , and vice versa for  $D_X$  and  $F$ . To further regularize the mappings, we introduce two *cycle consistency losses* that capture the intuition that if we translate from one domain to the other and back again we should arrive at where we started: (b) forward cycle-consistency loss:  $x \rightarrow G(x) \rightarrow F(G(x)) \approx x$ , and (c) backward cycle-consistency loss:  $y \rightarrow F(y) \rightarrow G(F(y)) \approx y$

# Reading

---

A. Brock, J. Donahue, and K. Simonyan. Large scale GAN training for high fidelity natural image synthesis. CoRR, abs/1809.11096, 2018.

V. Dumoulin, J. Shlens, and M. Kudlur. A learned representation for artistic style. CoRR, abs/1610.07629, 2016.

P. Isola, J. Zhu, T. Zhou, and A. A. Efros. Image-to-image translation with conditional adversarial networks. CoRR, abs/1611.07004, 2016.

J. Johnson, A. Alahi, and L. Fei-Fei. Perceptual losses for real-time style transfer and super-resolution. In European Conference on Computer Vision (ECCV), 2016.

X. Mao, Q. Li, H. Xie, R. Lau, Z. Wang, and S. Smolley. Least squares generative adversarial networks. CoRR, abs/1611.04076, 2016.

M. Mirza and S. Osindero. Conditional generative adversarial nets. CoRR, abs/1411.1784, 2014.

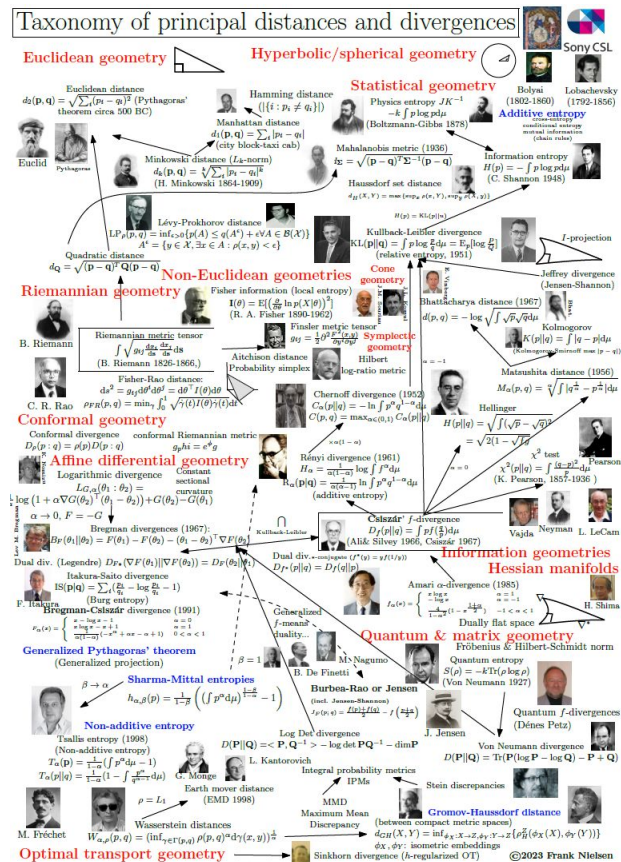
A. Radford, L. Metz, and S. Chintala. Unsupervised representation learning with deep convolutional generative adversarial networks. CoRR, abs/1511.06434, 2015.

A. Shrivastava, T. Pfister, O. Tuzel, J. Susskind, W. Wang, and R. Webb. Learning from simulated and unsupervised images through adversarial training. CoRR, abs/1612.07828, 2016.

J. Zhu, T. Park, P. Isola, and A. Efros. Unpaired image-to-image translation using cycle-consistent adversarial networks. CoRR, abs/1703.10593, 2017.

# Reading

<https://franknielsen.github.io/Divergence/>





# Reading

